Sprint 04 - React Events and Form Inputs.

0. Where You Are Before Sprint 04 (Sprint 04 ට පෙර ඔබ සිටිය යුතු තැන)

- **Concept Explanation (Sinhala):** Sprint 04 ආරම්භ කිරීමට පෙර, ඔබ React වල මූලික සංකල්ප කිහිපයක් දැන සිටිය යුතුයි:
  - React කියන්නේ User Interfaces (UI) හදන්න ගන්නා JavaScript library එකක්.
  - React App එකක් හදන්නේ නැවත නැවත භාවිත කළ හැකි කුඩා කොටස් වන Components එකතු කරලයි.
  - JSX මඟින් අපිට JavaScript ඇතුළේ HTML වගේ UI ලියන්න පුළුවන්.
  - Components කියන්නේ සරලවම JSX return කරන JavaScript functions.
  - Parent Component එකක ඉඳන් Child Component එකකට data යවන්න Props භාවිතා කරනවා.
  - `children` prop එකෙන් එක Component එකක් ඇතුළේ තවත් JSX wrap කරන්න පුළුවන්.
  - Array එකක තියෙන items list එකක් UI එකේ පෙන්නන්න `map()` method එක උදව් වෙනවා.
  - List එකක තියෙන items හරියට track කරගන්න React වලට stable `key` එකක් අවශ්‍ය වෙනවා.
  - Inline Styles ලියන්න React වලදී JavaScript Objects භාවිතා කරනවා.
  - Accessible JSX එකක් හදන්න semantic elements (e.g. `<main>`, `<button>`), නිවැරදි labels සහ `alt` attributes පාවිච්චි කරන්න ඕනේ.

  දැන් ඔබ static (වෙනස් නොවන) React screens හදන්න දන්නවා. හැබැයි තවමත් UI එකත් එක්ක interact කරන්න (ගනුදෙනු කරන්න) දන්නේ නැහැ.

  Sprint 04 වලින් අපි ඉගෙන ගන්නේ user කෙනෙක් button එකක් click කරද්දී, input එකක type කරද්දී හෝ form එකක් submit කරද්දී React UI එක රට ප්‍රතිචාර දක්වන්නේ කොහොමද කියන එකයි.

1. Sprint 04 Goal (Sprint 04 හි ඉලක්කය)

- **Concept Explanation (Sinhala):** මේ Sprint එකේ ප්‍රධානම අරමුණ වෙන්නේ React වලදී user actions (පරිශීලක ක්‍රියාකාරකම්) නිවැරදිව handle කරන්න ඉගෙන ගැනීමයි. මේ Sprint එක අවසානයේදී ඔබට:
  - Button clicks handle කරන්න,
  - Inputs වල type කරන ඒවා (typing) handle කරන්න,
  - Form submission handle කරන්න,
  - Form එකක් submit වෙද්දී මුළු page එකම refresh වෙන එක නවත්වන්න,
  - Controlled inputs හදන්න,
  - සරල Uncontrolled inputs හදන්න,
  - Form data validate (නිවැරදිදැයි පරීක්ෂා) කරන්න,
  - User ට feedback (error messages/success messages) පෙන්වන්න,
  - `preventDefault()` නිවැරදිව භාවිතා කරන්න,
  - අවශ්‍ය තැන්වලදී `stopPropagation()` පාවිච්චි කරන්න,
  - Event handlers වලට අමතර data (extra data) pass කරන්න,
  - Event සහ Form වලදී සිදුවන පොදු වැරදි හඳුනාගෙන ඒවා fix කරන්න පුළුවන් වෙනවා.

2. Big Mental Model (ප්‍රධාන මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** React වැඩ කරන්නේ කොහොමද කියලා ඔළුවට ගන්න පුළුවන් සරලම ක්‍රමය මෙන්න:

  Before Sprint 04 (පෙර තිබූ තත්ත්වය):

  ```
  Data (Props) තියෙනවා ➡️ React මඟින් UI එක Display කරනවා
  ```

  After Sprint 04 (දැන් පවතින තත්ත්වය):

  ```
  User යමක් කරයි (e.g. click) ➡️ React Event Handler එක run වෙයි ➡️
  State එක update වෙයි ➡️ React නැවත Render වී UI එක update කරයි
  ```

  උදාහරණයක් ලෙස Input Typing Cycle එක:

  ```
  User input එකක type කරයි ➡️ onChange event එක run වෙයි ➡️ setName()
  මඟින් State එක update කරයි ➡️ React re-render වෙයි ➡️ අලුත් value එක
  screen එකේ පෙන්වයි
  ```

3. What Is an Event? (Event එකක් කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala):** Event එකක් කියන්නේ browser එක ඇතුළේ සිදුවෙන ඕනෑම ක්‍රියාවකටයි. සාමාන්‍යයෙන් මේවා සිදුවෙන්නේ user ගේ ක්‍රියාකාරකම් නිසයි.
  - Button එකක් click කිරීම
  - Input box එකක type කිරීම
  - Form එකක් submit කිරීම
  - Select dropdown එකකින් option එකක් තේරීම
  - Checkbox එකක් check කිරීම
  - Card එකක් ඇතුළේ click කිරීම
  - Textarea එකක අකුරු වෙනස් කිරීම

4. React Event Names Use camelCase (Event නම් සඳහා camelCase භාවිතය)

- **Concept Explanation (Sinhala):** සාමාන්‍ය HTML වලදී event නම් ලියන්නේ lowercase (සියල්ල කුඩා අකුරෙන්). නමුත් React JSX වලදී අපි අනිවාර්යයෙන්ම **camelCase** (පළමු වචනය කුඩා අකුරෙන් සහ ඊළඟ වචනවල පළමු අකුර කැපිටල් අකුරෙන්) භාවිතා කළ යුතුයි.
  - HTML වල `onclick` ➡️ React වලදී `onClick`
  - HTML වල `onchange` ➡️ React වලදී `onChange`
  - HTML වල `onsubmit` ➡️ React වලදී `onSubmit`
- **Code Block:**

```
// HTML වල ලියන ආකාරය:
<button onclick="doSomething()">Click</button>

// React JSX වල ලියන නිවැරදි ආකාරය:
<button onClick={handleClick}>Click</button>
```

5. Event Handlers (Event Handler එකක් යනු කුමක්ද?)

- **Concept Explanation (Sinhala):** Event handler එකක් කියන්නේ කිසියම් event එකක් සිදුවූ විට run වීමට අප ලියන JavaScript function එකකටයි. සාමාන්‍යයෙන් අප මුලින්ම function එක ලියා, පසුව එය අදාළ JSX element එකට සම්බන්ධ කරනු ලබනවා.
- **Code Block:**

```
export default function App() {
  function handleClick() {
    console.log("Button clicked");
  }

  return (
    <main>
      <h1>Events Practice</h1>
      <button type="button" onClick={handleClick}>
        Click me
      </button>
    </main>
  );
}
```

6. Pass the Function, Do Not Call It Immediately (Function එක Pass කරන්න, Render වෙද්දීම Call කරන්න එපා)

- **Concept Explanation (Sinhala):** React වලදී සිදුවන ලොකුම සහ ප්‍රධානම වැරැද්දක් තමයි event එකක් දෙද්දී function එක පිටින්ම call කිරීම.
  - `onClick={handleClick}` ➡️ **නිවැරදියි.** (මෙහිදී අපි කරන්නේ function එකේ reference එක React වලට දෙන එකයි. එවිට user click කළ විට පමණක් එය run වේ).
  - `onClick={handleClick()}` ➡️ **වැරදියි.** (මෙහිදී component එක render වන මොහොතේදීම function එක run වී අවසන් වේ. User click කිරීමට පෙරම මෙය ක්‍රියාත්මක වේ).

7. Naming Event Handlers (Event Handlers වලට නම් තැබීම)

- **Concept Explanation (Sinhala):** Event handlers වලට නම් තැබීමේදී `handle` යන වචනයෙන් ආරම්භ කිරීම හොඳ පුරුද්දක් (naming convention). මෙය React වල අනිවාර්ය නීතියක් නොවුණත්, කේතය කියවීමේ පහසුව සඳහා භාවිත කරයි.
  - `handleClick`
  - `handleSubmit`
  - `handleChange`
  - `handleDelete`
- **Code Block:**

```
function handleClick() {
  console.log("Clicked");
}

<button onClick={handleClick}>Click</button>
```

8. Passing Extra Data to an Event Handler (Event Handler එකකට අමතර Data යැවීම)

- **Concept Explanation (Sinhala):** සමහර වෙලාවට අපිට event handler එකකට item ID එකක් වගේ අමතර data එකක් pass කරන්න අවශ්‍ය වෙනවා. එතකොට කෙලින්ම `onClick={handleBuy(id)}` කියලා ලිව්වොත් එය render වෙද්දීම run වෙනවා.

  මේක වළක්වන්න අපි **inline arrow function** එකක් ඇතුළේ අදාළ function එක wrap කරන්න ඕනේ: `onClick={() => handleBuy(id)}`. එවිට click කළ විට පමණක් එම arrow function එක හරහා අපේ function එක run වේ.
- **Code Block:**

```
function ProductCard({ id, title }) {
  function handleBuy(productId) {
    console.log("Buying product:", productId);
  }

  return (
    <article>
      <h2>{title}</h2>
      <button type="button" onClick={() => handleBuy(id)}>
        Buy
      </button>
    </article>
  );
}
```

9. Inline Arrow Handlers (JSX තුළම ලියන Arrow Handlers)

- **Concept Explanation (Sinhala):** කුඩා සහ සරල ක්‍රියාවන් සඳහා JSX එක ඇතුළේම කෙලින්ම arrow function එකක් ලියන්න පුළුවන්. හැබැයි ලොකු logic එකක් (පේළි කිහිපයක කේතයක්) තියෙනවා නම්, එය වෙනම named function එකක් විදිහට ලියන එක තමයි හොඳම ක්‍රමය.
- **Code Block:**

```
// සරල inline handler එකක්:
<button onClick={() => console.log("Deleted item", item.id)}>
  Delete
</button>
```

10. The Event Object (Event Object එක)

- **Concept Explanation (Sinhala):** Event එකක් සිදුවෙද්දී, React මඟින් අපේ handler function එකට automatic ම `event` කියන object එක ලබා දෙනවා. මේ object එක ඇතුළේ ඒ සිදුවූ event එකට අදාළ සියලුම තොරතුරු අඩංගුයි.

  විශේෂයෙන්ම input fields වලදී `event.target.value` කියන එකෙන් අදහස් වෙන්නේ "මේ වෙනස් වෙච්ච input box එකේ දැනට තියෙන අගය (value) ලබා ගන්න" යන්නයි.
- **Code Block:**

```
export default function App() {
  function handleChange(event) {
    console.log(event.target.value);
  }

  return (
    <main>
      <label htmlFor="name">Name</label>
      <input id="name" type="text" onChange={handleChange} />
    </main>
  );
}
```

11. Synthetic Events (Synthetic Events යනු මොනවාද?)

- **Concept Explanation (Sinhala):** React විසින් browser එකේ සාමාන්‍ය native events කෙලින්ම පාවිච්චි නොකර, ඒවා `SyntheticEvent` කියන wrapper එකකින් ඔතා අපට ලබා දෙයි.

  මෙසේ කරන්නේ විවිධ browser (Chrome, Safari, Firefox) වල events එකම විදිහට, කිසිදු වෙනසක් නැතිව වැඩ කරන බව තහවුරු කර ගැනීමටයි (cross-browser consistency). ආරම්භකයෙක් විදිහට මෙහි ගැඹුරු සිද්ධාන්ත මතක තබා ගැනීමට අවශ්‍ය නොවන අතර, event object එකෙන් අපිට අවශ්‍ය දත්ත ලබාගත හැකි බව දැන සිටීම ප්‍රමාණවත්ය.

12. onChange for Inputs (Inputs සඳහා onChange භාවිතය)

- **Concept Explanation (Sinhala):** Input field එකක user යමක් type කරද්දී, delete කරද්දී හෝ වෙනස් කරද්දී ඒ හැම වෙනසකටම ප්‍රතිචාර දක්වන්න `onChange` event එක පාවිච්චි කරනවා.
  - State එක live update කරන්න,
  - Live preview එකක් පෙන්වන්න,
  - Field එක validate කරන්න,
  - Character count එකක් live පෙන්වන්න `onChange` ප්‍රයෝජනවත් වේ.

13. Controlled Inputs (Controlled Inputs යනු මොනවාද?)

- **Concept Explanation (Sinhala):** React වලදී form inputs පාවිච්චි කරන ප්‍රධානම සහ වඩාත්ම නිර්දේශිත ක්‍රමය තමයි **Controlled Inputs**. Controlled Input එකක් කියන්නේ "එහි value එක සම්පූර්ණයෙන්ම React State එකක් මඟින් පාලනය වන input එකක්".

  Controlled input එකක් හදන්න කරුණු 4ක් අවශ්‍යයි:
  1. State variable එකක් (`const [name, setName] = useState("")`)
  2. State එක වෙනස් කරන setter function එකක් (`setName`)
  3. Input එකේ `value` attribute එකට state variable එක දීම (`value={name}`)
  4. Input එකේ `onChange` එකට state එක update කරන handler එක දීම (`onChange={(e) => setName(e.target.value)}`)
- **Code Block:**

```
import { useState } from "react";

export default function App() {
  const [name, setName] = useState("");

  return (
    <main>
      <h1>Controlled Input</h1>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        type="text"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
      <p>You typed: {name}</p>
    </main>
  );
}
```

14. Controlled Input Cycle (Controlled Input එකක් ක්‍රියා කරන ආකාරය)

- **Concept Explanation (Sinhala):** Controlled input එකක දත්ත ගලායන ආකාරය (data flow) මෙසේය:

  ```
  User අකුරක් type කරයි ➡️ onChange event එක run වෙයි ➡️ setName() මඟින්
  React state එක update වෙයි ➡️ React component එක re-render වෙයි ➡️
  Input box එකේ අලුත් state value එක පෙන්නුම් කරයි
  ```

  උදාහරණයක් ලෙස, පරිශීලකයා "S" අකුර type කළ විට, React විසින් state එක "S" වලට වෙනස් කර, මුළු component එකම re-render කර input එකේ value එක "S" බවට පත් කරයි.

15. Why Controlled Inputs Are Useful (Controlled Inputs වල ඇති වාසි)

- **Concept Explanation (Sinhala):** Controlled inputs වලදී React සතුව සැමවිටම input එකේ පවතින අලුත්ම අගය (state) පවතී. එම නිසා:
  - Live character count පෙන්වීම,
  - Live validation සිදු කිරීම,
  - Form එකක් submit කළ පසු inputs පහසුවෙන්ම clear/reset කිරීම,
  - දත්ත වැරදි නම් submit button එක disable කිරීම වැනි දේ පහසුවෙන්ම කළ හැක.
- **Code Block:**

```
import { useState } from "react";

export default function MessageBox() {
  const [message, setMessage] = useState("");

  return (
    <main>
      <h1>Message Box</h1>
      <label htmlFor="message">Message</label>
      <textarea
        id="message"
        value={message}
        onChange={(event) => setMessage(event.target.value)}
      />
      <p>Characters: {message.length}</p>
    </main>
  );
}
```

16. Form Submission in React (React වලදී Form එකක් Submit කිරීම)

- **Concept Explanation (Sinhala):** Form එකක් submit වෙන්නේ පරිශීලකයා submit button එකක් click කළ විට හෝ input එකක් ඇතුළේ සිට Enter key එක එබූ විටයි.

  සාමාන්‍ය HTML වලදී form එකක් submit වෙද්දී මුළු page එකම reload/refresh වේ. නමුත් Single Page Application (SPA) එකක් වන React වලදී අපිට මුළු page එකම refresh කිරීමට අවශ්‍ය නැත. එම නිසා page reload වීම වැළැක්වීමට අපි `event.preventDefault()` භාවිතා කරමු.

17. event.preventDefault()

- **Concept Explanation (Sinhala):** `event.preventDefault()` මඟින් browser එකේ තිබෙන සාමාන්‍ය default හැසිරීම (default behavior) නවත්වනු ලබයි. Form එකක් submit වීමේදී සිදුවන page refresh එක නවත්වන්නේ මෙයිනි.
- **Code Block:**

```
import { useState } from "react";

export default function SignupForm() {
  const [name, setName] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    console.log("Submitted name:", name);
  }

  return (
    <main>
      <h1>Signup</h1>
      <form onSubmit={handleSubmit}>
        <label htmlFor="name">Name</label>
        <input
          id="name"
          type="text"
          value={name}
          onChange={(event) => setName(event.target.value)}
        />
        <button type="submit">Submit</button>
      </form>
    </main>
  );
}
```

18. onSubmit Belongs on the Form (onSubmit එක Form element එකටම ලබාදෙන්න)

- **Concept Explanation (Sinhala):** බොහෝ ආරම්භකයින් කරන වැරැද්දක් තමයි submit button එකේ `onClick` එකට submit කරන logic එක දීම.

  නමුත් නිවැරදි ක්‍රමය වන්නේ `<form>` tag එකේ `onSubmit={handleSubmit}` ලෙස ලබා දී, button එකේ `type="submit"` ලෙස තැබීමයි. එවිට button එක click කිරීමෙන් මෙන්ම, input box එකක් මත සිට Enter key එක එබීමෙන්ද form එක සාර්ථකව submit වේ.

19. Button Types (Button වලට වර්ග ලබාදීම)

- **Concept Explanation (Sinhala):** Form එකක් ඇතුළේ තියෙන buttons වලට නිවැරදි `type` එක ලබා දීම අනිවාර්ය වේ:
  - Form එක submit කරන්න: `<button type="submit">Submit</button>`
  - සාමාන්‍ය වෙනත් ක්‍රියාවක් කරන්න (e.g. Cancel): `<button type="button">Cancel</button>`

  සටහන: Form එකක් ඇතුළේ සාමාන්‍ය `<button>` එකක් (type එක සඳහන් නොකර) තැබුවහොත්, එය browser එක මඟින් automatic ම submit button එකක් ලෙස සලකා ක්‍රියා කරයි. මෙය වැළැක්වීමට අදාළ type එක නිවැරදිව සඳහන් කරන්න.

20. Form Validation (Form දත්ත නිවැරදිදැයි පරීක්ෂා කිරීම)

- **Concept Explanation (Sinhala):** Validation කියන්නේ පරිශීලකයා ඇතුළත් කළ දත්ත අප බලාපොරොත්තු වන මට්ටමේ තිබේදැයි පරීක්ෂා කිරීමයි (e.g. name field එක හිස් නොවීම, password එක දිගින් වැඩි වීම).

  `name.trim().length > 0` මඟින් පරිශීලකයා හිස්තැන් (spaces) පමණක් type කර submit කිරීම වළක්වයි.
- **Code Block:**

```
import { useState } from "react";

export default function SignupForm() {
  const [name, setName] = useState("");
  const nameIsValid = name.trim().length > 0;

  function handleSubmit(event) {
    event.preventDefault();

    if (!nameIsValid) {
      console.log("Name is required");
      return; // form එක submit වීම මෙතනින් නවතී.
    }

    console.log("Submitted:", name);
  }

  return (
    <main>
      <h1>Signup</h1>
      <form onSubmit={handleSubmit}>
        <label htmlFor="name">Name</label>
        <input
          id="name"
          type="text"
          value={name}
          onChange={(event) => setName(event.target.value)}
        />
        {!nameIsValid && <p style={{ color: 'red' }}>Name is required.
</p>}
        <button type="submit" disabled={!nameIsValid}>
          Submit
        </button>
      </form>
    </main>
  );
}
```

21. disabled Attribute (disabled ගුණය)

- **Concept Explanation (Sinhala):** `disabled` attribute එක මඟින් form එකක් invalid (අසම්පූර්ණ) අවස්ථාවක පවතින විට, submit button එක click කිරීමට නොහැකි වන සේ අක්‍රිය කර තැබිය හැක.
  - `formIsValid` එක `false` නම් ➡️ button එක disable වේ.
  - `formIsValid` එක `true` නම් ➡️ button එක enable වේ.

22. Multiple Controlled Inputs (Controlled Inputs කිහිපයක් එකම Form එකක භාවිතය)

- **Concept Explanation (Sinhala):** සැබෑ ලෝකයේ form එකක inputs කිහිපයක් පවතී. ඒ සඳහා අපිට වෙන වෙනම state variables කිහිපයක් භාවිතා කළ හැක.

  පහත දැක්වෙන්නේ text input, select dropdown සහ textarea යන සියල්ලම එක් කර සාදන ලද FeedbackForm එකකි. Form එක submit වූ පසු state variables හිස් අගයන්ට පත් කිරීමෙන් මුළු form එකම reset කරනු ලබයි.
- **Code Block:**

```
import { useState } from "react";

export default function FeedbackForm() {
  const [name, setName] = useState("");
  const [rating, setRating] = useState("5");
  const [message, setMessage] = useState("");

  const formIsValid = name.trim() !== "" && message.trim() !== "";

  function handleSubmit(event) {
    event.preventDefault();

    if (!formIsValid) {
      return;
    }

    const feedbackData = {
      name,
      rating,
      message,
    };

    console.log(feedbackData);

    // Form එක reset කිරීම:
    setName("");
    setRating("5");
    setMessage("");
  }

  return (
    <main>
      <h1>Feedback Form</h1>
      <form onSubmit={handleSubmit}>
        <div>
          <label htmlFor="name">Name</label>
          <input
            id="name"
            type="text"
            value={name}
            onChange={(event) => setName(event.target.value)}
          />
        </div>

        <div>
          <label htmlFor="rating">Rating</label>
          <select
            id="rating"
            value={rating}
            onChange={(event) => setRating(event.target.value)}
          >
            <option value="5">5 - Excellent</option>
            <option value="4">4 - Good</option>
            <option value="3">3 - Okay</option>
            <option value="2">2 - Poor</option>
            <option value="1">1 - Bad</option>
          </select>
        </div>

        <div>
          <label htmlFor="message">Message</label>
          <textarea
            id="message"
            value={message}
            onChange={(event) => setMessage(event.target.value)}
          />
        </div>

        <p>Message length: {message.length}</p>

        <button type="submit" disabled={!formIsValid}>
          Send Feedback
        </button>
      </form>
    </main>
  );
}
```

23. Controlled textarea (Controlled textarea භාවිතය)

- **Concept Explanation (Sinhala):** React වලදී `<textarea>` එකක් සාමාන්‍ය controlled input එකක් ලෙසම ක්‍රියා කරයි. HTML වලදී මෙන් `<textarea>Hello</textarea>` ලෙස ඇතුළත අගයන් නොලියා, React වලදී `value` සහ `onChange` attributes භාවිතා කළ යුතුය.

24. Controlled select (Controlled select භාවිතය)

- **Concept Explanation (Sinhala):** `<select>` element එකකද තෝරාගෙන ඇති අගය (selected option) පාලනය කරන්නේ එහි `value` attribute එකට දෙන state variable එක මඟිනි. පරිශීලකයා වෙනත් option එකක් තේරූ විට `onChange` handler එක මඟින් state එක update කරනු ලබයි.

25. Resetting a Controlled Form (Controlled Form එකක් Reset කිරීම)

- **Concept Explanation (Sinhala):** Controlled inputs වල අගයන් පවතින්නේ React state එකේ නිසා, form එකක් reset කිරීමට නම් කළ යුත්තේ අදාළ state variables හිස් කිරීමයි (e.g., `setName("")`). State එක හිස් වූ වහාම React re-render වී UI එකේ ඇති inputs ද හිස් වේ.

26. Uncontrolled Inputs (Uncontrolled Inputs යනු මොනවාද?)

- **Concept Explanation (Sinhala):** Uncontrolled Input එකක් කියන්නේ "එහි අගය (value) React state එක මඟින් පාලනය නොවී, කෙලින්ම DOM (Document Object Model) එක ඇතුළේම පාලනය වන input එකක්".
  - **Controlled:** අගය පවතින්නේ React State එකේය.
  - **Uncontrolled:** අගය පවතින්නේ DOM එකේ (browser එකේ) ය.

27. useRef for Uncontrolled Inputs (Uncontrolled Inputs සඳහා useRef භාවිතය)

- **Concept Explanation (Sinhala):** Uncontrolled input එකක ඇති දත්ත කියවා ගැනීමට (read කිරීමට) අපි `useRef` කියන Hook එක භාවිතා කරමු. මෙහිදී input එක type වෙද්දී state update නොවන අතර, submit කරන මොහොතේදී පමණක් `current.value` මඟින් අගය කියවා ගනු ලබයි.
- **Code Block:**

```
import { useRef } from "react";

export default function RefCodeForm() {
  const refCodeInput = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();
    const refCode = refCodeInput.current.value;
    console.log("Referral code:", refCode);
  }

  return (
    <main>
      <h1>Referral Code</h1>
      <form onSubmit={handleSubmit}>
        <label htmlFor="refCode">Referral Code</label>
        <input id="refCode" type="text" ref={refCodeInput} />
        <button type="submit">Submit</button>
      </form>
    </main>
  );
}
```

28. When Are Uncontrolled Inputs Reasonable? (Uncontrolled Inputs වඩාත් සුදුසු අවස්ථා)

- **Concept Explanation (Sinhala):** පහත අවස්ථාවන්හිදී Uncontrolled inputs භාවිතා කිරීම ගැටලුවක් නොවේ:
  - දත්ත අවශ්‍ය වන්නේ form එක submit වන මොහොතේදී පමණක් නම්.
  - Live validation අවශ්‍ය නොවේ නම්.
  - Live character preview වැනි දේ අවශ්‍ය නොවේ නම්.
  - React නොවන වෙනත් බාහිර library එකක් සමඟ input එක සම්බන්ධ කිරීමට අවශ්‍ය නම්.

  කෙසේ වෙතත්, සාමාන්‍ය React වැඩ වලදී සැමවිටම **Controlled inputs** භාවිතා කිරීම වඩාත් නිර්දේශිතය.

29. Controlled vs Uncontrolled Inputs (Controlled සහ Uncontrolled සංසන්දනය)

- **Concept Explanation (Sinhala):**

කරුණ (Topic)	Controlled Input	Uncontrolled Input
Value එක තැන්පත් වන්නේ	React state එකේ	DOM input එකේ
`value` attribute එක ගන්නවාද?	ඔව්	සාමාන්‍යයෙන් නැත
`onChange` භාවිතා කරනවාද?	ඔව්	සැමවිටම නැත
`useRef` භාවිතා කරනවාද?	සාමාන්‍යයෙන් නැත	බොහෝ විට ඔව්
Live Validation වලට සුදුසුද?	ඔව්	සුදුසු නැත
Live Preview එකක් ගැනීමට සුදුසුද?	ඔව්	සුදුසු නැත
Submit එකේදී පමණක් අගය ගැනීමට	කළ හැක	ඉතා සුදුසුයි

30. event.stopPropagation() (Event එකක් ඉහළට ගලා යාම නැවැත්වීම)

- **Concept Explanation (Sinhala):** Browser එකේ element එකක් ඇතුළේ click එකක් කළ විට, එම click event එක එහි parent element (මව් element) එක දක්වා ඉහළට ගමන් කරයි. මෙයට **Event Bubbling** යැයි කියනු ලැබේ.

  උදාහරණයකට Card එකක් ඇතුළේ Button එකක් තිබෙන විට, button එක click කළහොත් button click handler එක මෙන්ම card click handler එකද run විය හැක. මෙය වළක්වා event එක ඉහළට යාම නැවැත්වීමට `event.stopPropagation()` භාවිතා කරමු.
- **Code Block:**

```
export default function App() {
  function handleCardClick() {
    console.log("Card clicked");
  }

  function handleButtonClick(event) {
    event.stopPropagation(); // Event එක card එක දක්වා ඉහළට යාම නවත්වයි.
    console.log("Button clicked");
  }

  return (
    <article onClick={handleCardClick} style={{ padding: '20px', border:
'1px solid black' }}>
      <h2>Product</h2>
      <button type="button" onClick={handleButtonClick}>
        Buy
      </button>
    </article>
  );
}
```

31. preventDefault() vs stopPropagation() (සංසන්දනය)

- **Concept Explanation (Sinhala):** | Method | නවත්වනු ලබන ක්‍රියාව (What it stops) | පොදු භාවිතය (Common use) | | :--- | :--- | :--- | | `preventDefault()` | Browser එකේ default ක්‍රියාකාරකම | Form එකක් submit වෙද්දී page reload වීම වැළැක්වීම | | `stopPropagation()` | Event එක ඉහළට ගලා යාම (Bubbling) | Child element එකක් click වන විට parent element එකේ click event එක run වීම වැළැක්වීම |

32. Accessibility Habits for Forms (Forms සඳහා Accessibility පුරුදු)

- **Concept Explanation (Sinhala):** - සැබෑ HTML form elements (`<form>`, `<input>`, `<textarea>`, `<select>`) ම භාවිතා කරන්න.
  - Inputs සමඟ labels අනිවාර්යයෙන් සම්බන්ධ කරන්න.
  - JSX වලදී `for` යනු reserved keyword එකක් නිසා label එක input එක හා සම්බන්ධ කිරීමට `htmlFor` භාවිතා කරන්න. (label එකේ `htmlFor` අගය සහ input එකේ `id` අගය සමාන විය යුතුය).
  - Clickable `div` elements වෙනුවට සැබෑ `<button>` භාවිතා කරන්න.
- **Code Block:**

```
// නිවැරදි ක්‍රමය:
<label htmlFor="email">Email</label>
<input id="email" type="email" />

// වැරදි ක්‍රමය (JSX වල 'for' ලිවිය නොහැක):
<label for="email">Email</label>
```

33. Common Sprint 04 Mistakes (සිදුවන පොදු වැරදි 7ක්)

- **Concept Explanation (Sinhala):** 1. **Lowercase event names ලිවීම:** `onchange` වෙනුවට `onChange` ද, `onsubmit` වෙනුවට `onSubmit` ද ලෙස camelCase භාවිතා කළ යුතුය.

2. **Event handler එක render වෙද්දීම call කිරීම:** `onClick={handleClick()}` වෙනුවට `onClick={handleClick}` ලෙස reference එක පමණක් ලබා දිය යුතුය.
3. **Arrow function එකක් නැතිව data pass කිරීම:** `onClick={handleDelete(id)}` වෙනුවට `onClick={() => handleDelete(id)}` ලෙස ලියන්න.
4. `preventDefault()` **අමතක වීම:** Form submit එකකදී page එක refresh වීම වැළැක්වීමට මෙය අනිවාර්ය වේ.
5. **පවතින්නේ කොහේදැයි නොදැන `e` හෝ `event` භාවිතා කිරීම:** `onChange={setName(e.target.value)}` ලෙස ලිවීම වැරදිය. නිවැරදි ක්‍රමය: `onChange={(event) => setName(event.target.value)}` වේ.
6. **Controlled input එකක `value` attribute එක අමතක වීම:** `onChange` පමණක් ලියා `value={state}` ලිවීම අතපසු නොකරන්න.
7. **Label එක input එකට සම්බන්ධ නොකිරීම:** Accessibility සඳහා `htmlFor` and `id` හරියට සම්බන්ධ කරන්න.

34. Debug Task From Sprint 04 (වැරදි සහිත කේතය නිවැරදි කිරීම)

- **Concept Explanation (Sinhala):** පහත දැක්වෙන්නේ වැරදි සහිතව තිබූ කේතයක් (Broken Code) සියලු වැරදි නිවැරදි කර සකස් කරන ලද ආකාරයයි.

  තිබූ වැරදි:
  1. `useState` import කර නොතිබීම.
  2. `onsubmit` ලෙස lowercase ලිවීම (`onSubmit` විය යුතුය).
  3. `submit()` ලෙස සෘජුවම call කර තිබීම (`onSubmit={handleSubmit}` විය යුතුය).
  4. `onchange` ලෙස lowercase ලිවීම (`onChange` විය යුතුය).
  5. Arrow function එකකින් තොරව `e` භාවිතා කර තිබීම (`(event) => setName(event.target.value)` විය යුතුය).
  6. `preventDefault()` භාවිතා නොකිරීම නිසා page reload වීම.
- **Code Block:**

```
// සම්පූර්ණයෙන්ම නිවැරදි කරන ලද කේතය (Fixed Version):
import { useState } from "react";

export default function App() {
  const [name, setName] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    console.log(name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```

35. Full Practice Project: Contact Us Form (ප්‍රායෝගික ව්‍යාපෘතිය)

- **Concept Explanation (Sinhala):** මෙම Contact Us Form එක මඟින් name, email සහ message යන fields තුනම controlled inputs ලෙස පාලනය කරයි. Message එකෙහි character count එක live පෙන්වන අතර සියලු fields පිරෙන තෙක් submit button එක disable කර තබයි. Submit වූ පසු form එක reset වේ.
- **Code Block:**

```
import { useState } from "react";

export default function ContactForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");

  const formIsValid =
    name.trim() !== "" &&
    email.trim() !== "" &&
    message.trim() !== "";

  function handleSubmit(event) {
    event.preventDefault();

    if (!formIsValid) {
      return;
    }

    console.log({
      name,
      email,
      message,
    });

    // Reset fields after submit:
    setName("");
    setEmail("");
    setMessage("");
  }

  return (
    <main style={{ padding: '20px', maxWidth: '400px', margin: '0 auto'
}}>
      <h1>Contact Us</h1>
      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '10px' }}>
          <label htmlFor="name">Name</label>
          <input
            id="name"
            type="text"
            value={name}
            onChange={(event) => setName(event.target.value)}
            style={{ width: '100%', padding: '8px', marginTop: '5px' }}
          />
        </div>

        <div style={{ marginBottom: '10px' }}>
          <label htmlFor="email">Email</label>
          <input
            id="email"
            type="email"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
            style={{ width: '100%', padding: '8px', marginTop: '5px' }}
          />
        </div>

        <div style={{ marginBottom: '10px' }}>
          <label htmlFor="message">Message</label>
          <textarea
            id="message"
            value={message}
            onChange={(event) => setMessage(event.target.value)}
            style={{ width: '100%', padding: '8px', marginTop: '5px',
height: '100px' }}
          />
        </div>

        <p>Characters: {message.length}</p>

        <button
          type="submit"
          disabled={!formIsValid}
          style={{ padding: '10px 20px', cursor: formIsValid ? 'pointer'
: 'not-allowed' }}
        >
          Send
        </button>
      </form>
    </main>
  );
}
```

36. Guided Practice: Feedback Form (starter structure)

- **Concept Explanation (Sinhala):** මෙම ආකෘතිය (starter structure) භාවිතා කර ඔබට තනිවම feedback form එකක් නිර්මාණය කර පුහුණු විය හැක.
- **Code Block:**

```
import { useState } from "react";

export default function FeedbackForm() {
  const [name, setName] = useState("");
  const [rating, setRating] = useState("5");
  const [message, setMessage] = useState("");

  const formIsValid = name.trim() !== "" && message.trim() !== "";

  function handleSubmit(event) {
    event.preventDefault();

    if (!formIsValid) {
      return;
    }

    console.log({ name, rating, message });

    setName("");
    setRating("5");
    setMessage("");
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* ඔබගේ inputs සහ labels මෙතනින් ඇතුළත් කරන්න */}
    </form>
  );
}
```

37. Challenge Practice: Add an Uncontrolled Referral Code (අභියෝගාත්මක අභ්‍යාසය)

- **Concept Explanation (Sinhala):** මෙම කේතයෙන් පෙන්වන්නේ Controlled input එකක් (name සඳහා) සහ Uncontrolled input එකක් (`useRef` මඟින් referral code සඳහා) එකම form එකක් තුළ එකට එකතු කර ක්‍රියා කරන ආකාරයයි.
- **Code Block:**

```
import { useRef, useState } from "react";

export default function ChallengeForm() {
  const [name, setName] = useState("");
  const refCodeInput = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();
    const refCode = refCodeInput.current.value;

    console.log({
      name,
      refCode,
    });
  }

  return (
    <form onSubmit={handleSubmit} style={{ padding: '20px' }}>
      <div style={{ marginBottom: '10px' }}>
        <label htmlFor="name">Name</label>
        <input
          id="name"
          value={name}
          onChange={(event) => setName(event.target.value)}
          style={{ display: 'block', margin: '5px 0' }}
        />
      </div>

      <div style={{ marginBottom: '10px' }}>
        <label htmlFor="refCode">Referral Code</label>
        <input
          id="refCode"
          ref={refCodeInput}
          style={{ display: 'block', margin: '5px 0' }}
        />
      </div>

      <button type="submit">Submit</button>
    </form>
  );
}
```

38. Sprint 04 Revision Questions and Answers (පුනරීක්ෂණ ප්‍රශ්නෝත්තර)

- **Concept Explanation (Sinhala): ප්‍රශ්නය 1: React event නම් සඳහා onclick වෙනුවට onClick (camelCase) ලියන්නේ ඇයි?**
  - **පිළිතුර:** JSX කියන්නේ JavaScript වලට සමාන syntax එකක් නිසා, JavaScript වල සම්මතය වන camelCase අකුරු රටාව React වලදී event නම් සඳහාද භාවිතා කරයි.

  ප්‍රශ්නය 2: `onClick={handleClick()}` වෙනුවට `onClick={handleClick}` ලිවිය යුත්තේ ඇයි?
  - **පිළිතුර:** `onClick={handleClick}` මඟින් function එකේ reference එක පමණක් React වෙත ලබා දෙන අතර, user click කළ විට පමණක් එය run වේ. නමුත් `onClick={handleClick()}` ලිව්වොත් component එක render වන මොහොතේදීම function එක run වී අවසන් වේ.

  ප්‍රශ්නය 3: Form එකක `event.preventDefault()` මඟින් කරන්නේ කුමක්ද?
  - **පිළිතුර:** Browser එකේ සිදුවන සාමාන්‍ය default හැසිරීම නවත්වයි. React form එකකදී මෙයින් සිදුවන්නේ form එක submit වෙද්දී page එක reload/refresh වීම වැළැක්වීමයි.

  ප්‍රශ්නය 4: `event.target.value` මඟින් අපට ලැබෙන්නේ කුමක්ද?
  - **පිළිතුර:** එම event එක සිදු වූ input field එකෙහි දැනට පවතින අලුත්ම අගය (typed value) අපට ලබා දෙයි.

  ප්‍රශ්නය 5: Controlled Input එකක් යනු කුමක්ද?
  - **පිළිතුර:** Input එකේ පවතින අගය (value) සම්පූර්ණයෙන්ම React state එකක් මඟින් පාලනය වන සහ තැන්පත් කර ගන්නා input එකකි.

  ප්‍රශ්නය 6: Controlled Inputs, validation සඳහා ඉතා ප්‍රයෝජනවත් වන්නේ ඇයි?
  - **පිළිතුර:** React සතුව සැමවිටම input එකෙහි පවතින අගය සජීවීව (live) පවතින නිසා, user type කරන අතරතුරදීම දත්ත නිවැරදිදැයි පරීක්ෂා කිරීමට, error messages පෙන්වීමට සහ buttons disable කිරීමට React වලට හැකියාව ලැබේ.

  ප්‍රශ්නය 7: Uncontrolled Input එකක් භාවිතා කිරීම වඩාත් සුදුසු වන්නේ කුමන අවස්ථාවකදීද?
  - **පිළිතුර:** අපට input එකෙහි අගය අවශ්‍ය වන්නේ form එක submit වන මොහොතේදී පමණක් නම් සහ කිසිදු live validation එකක් හෝ live preview එකක් අවශ්‍ය නොවේ නම් පමණක් Uncontrolled Input එකක් භාවිතා කිරීම සුදුසුය.

  ප්‍රශ්නය 8: Uncontrolled Input එකක අගය ලබා ගැනීමට `useRef` උදව් වන්නේ කෙසේද?
  - **පිළිතුර:** `useRef` මඟින් HTML DOM element එක වෙත සෘජු සම්බන්ධතාවයක් (reference) ලබා දෙන අතර, එමඟින් `inputRef.current.value` භාවිත කර ඕනෑම වෙලාවක එහි ඇති අගය කියවා ගත හැක.

  ප්‍රශ්නය 9: Form inputs සමඟ labels සම්බන්ධ කළ යුත්තේ ඇයි?
  - **පිළිතුර:** Accessibility (පහසුවෙන් භාවිත කිරීමේ හැකියාව) වැඩි දියුණු කිරීමට සහ screen readers භාවිතා කරන අයට කුමන input එක කුමකටදැයි පැහැදිලිව හඳුනා ගැනීමට එය උපකාරී වේ.

  ප්‍රශ්නය 10: Submit buttons සඳහා `type="submit"` භාවිතා කළ යුත්තේ ඇයි?
  - **පිළිතුර:** Browser එකට එය form එක submit කරන button එකක් බව පැහැදිලිව හඳුන්වා දීමටත්, form එකෙහි `onSubmit` handler එක නිවැරදිව ක්‍රියාත්මක කරවීමටත් එය උපකාරී වේ.